virvelle

CORSO DI MOBILE PROGRAMMING

PROGRAMMA GOL

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary

Finanziato dall'Unione europea
NextGenerationEU

Recap: Dove eravamo rimasti?

Bentornati! Nel Giorno 1 abbiamo visto:

- Il passaggio storico da **Java a Kotlin**.
- L'addio all'XML in favore di **Jetpack Compose**.
- I 3 mattoncini base della UI: `Row`, `Column`, `Box`.
- Cos'è un `Modifier` per stilizzare i componenti.

Oggi passiamo dalla teoria alla pratica vera: diamo un "cervello" alle nostre app.

Obiettivi di Oggi

Logica, Stato e Liste Dinamiche

1. **Il Cervello (Logica):** Costrutti moderni di Kotlin.
2. **I Dati (Liste):** Come gestire tante informazioni.
3. **La Memoria (State):** Insegnare all'app a ricordare.
4. **Le Liste Infinite:** Il componente `LazyColumn`.
5. **Il Progetto:** Costruiremo insieme una *To-Do List*.

virvelle

Parte 1: Il Cervello dell'App

Logica e controllo del flusso in Kotlin

L'Evoluzione dell' `if`

In Kotlin, l'if restituisce un valore

In vecchi linguaggi, l' `if` serviva solo a eseguire comandi. In Kotlin, può assegnare direttamente una variabile!

Sintassi tradizionale:

```
var stato: String
if (batteria > 20) {
    stato = "Acceso"
} else {
    stato = "Risparmio Energetico"
}
```

L' `if` come Espressione (Il modo Kotlin)

Risparmiamo righe di codice e rendiamo tutto più leggibile usando l' `if` direttamente nell'assegnazione:

Kotlin

```
val stato = if (batteria > 20) "Acceso" else "Risparmio"
```

È pulito, veloce e ci permette di usare `val` (costante) invece di `var`, rendendo il codice più sicuro.

Il Killer dello Switch: `when`

Dimenticate il vecchio "switch-case"

Se avete più di due opzioni, l'`if` diventa un incubo da leggere. Kotlin introduce `when`, uno strumento potentissimo.

Kotlin

```
val voto = 8

when (voto) {
    10 -> println("Perfetto!")
    8, 9 -> println("Ottimo lavoro")
    in 6..7 -> println("Sufficiente")
    else -> println("Devi studiare di più")
}
```

Notate come possiamo usare range (`in 6..7`) e liste di valori (`8, 9`)!

Assegnazione con il `when`

Proprio come l'`if`, anche il `when` può restituire un valore direttamente a una variabile. Questo è utilissimo nello sviluppo Android per decidere che schermata mostrare!

Kotlin

```
val coloreSfondo = when (temaSelezionato) {
    "Scuro" -> Color.Black
    "Chiaro" -> Color.White
    else -> Color.Gray
}
```

Dibattito: Sicurezza vs Flessibilità

Perché linguaggi moderni come Kotlin e Swift ci spingono a usare `val` (costanti) ovunque possibile al posto di `var` (variabili modificabili)?

Se un dato non cambia mai, perché dovremmo bloccarlo? Che vantaggio ci dà quando l'app diventa enorme o lavoriamo in un team di 10 persone?

virvelle

Parte 2: Lavorare con i Dati

Array e Collezioni in Kotlin

Dati Singoli vs Collezioni

Fino ad ora abbiamo gestito singoli dati: `val nome = "Alfonso"`

Ma un'app reale gestisce **liste di dati**:

- I messaggi su WhatsApp.
- Le foto su Instagram.
- I prodotti su Amazon.

Abbiamo bisogno delle **Collezioni**.

List vs MutableList

La regola d'oro di Kotlin

In Kotlin, le liste sono divise in due categorie strettamente separate:

1. `List` **(Sola Lettura):** Puoi leggerla, ma NON puoi aggiungere o rimuovere elementi. È sicura e immodificabile.
2. `MutableList` **(Modificabile):** Puoi alterarla nel tempo (aggiungere, rimuovere, svuotare).

virvelle

Come creare Liste

Lista in sola lettura (Sicura):

Kotlin

```
val giorni = listOf("Lunedì", "Martedì", "Mercoledì")
// giorni.add("Giovedì") <-- ERRORE! Non compila.
```

Lista Mutabile (Dinamica):

Kotlin

```
val toDos = mutableListOf("Fare la spesa", "Studiare Kotlin")
toDos.add("Chiamare Marco") // Questo funziona!
toDos.removeAt(0)           // Rimuove "Fare la spesa"
```

Come leggere i dati di una Lista?

Dimenticate il vecchio "for (i=0; i<N; i++)"

Kotlin ci regala il `forEach`, che cicla elegantemente su ogni elemento senza dover gestire numeri e indici.

Kotlin

```
val studenti = listOf("Mario", "Luigi", "Peach")

studenti.forEach { studente ->
    println("Buongiorno, $studente!")
}
```

virvelle

Mini-Quiz di Logica 🧠

Cosa stamperà questo blocco di codice?

Kotlin

```
val numeri = mutableListOf(1, 2, 3)
numeri.add(4)
numeri.remove(2)

println(numeri.size)
```

- A)4
- B)3
- C)2
- D) Errore di compilazione

(Pensate alla differenza tra l'oggetto rimosso e il suo indice!)

virvelle

Parte 3: Il Cuore di Compose

Lo "Stato" e la Ricomposizione

Il grande problema delle interfacce

Immaginate questo scenario:

1. Ho una variabile `var messaggiNonLetti = 2`.
2. Disegno l'interfaccia che mostra il numero "2".
3. Arriva un nuovo messaggio, quindi aggiorno la variabile: `messaggiNonLetti = 3`.

Domanda: *Lo schermo mostrerà 3?*

Risposta: NO! 💥

La variabile si aggiorna nella memoria del telefono, ma **lo schermo non lo sa**.

L'interfaccia utente è pigra: una volta disegnata, resta immobile finché non le diamo un motivo valido per ridisegnarsi.

Abbiamo bisogno di uno **Stato Osservabile**.

Lo Stato: Il "Cervello" della UI

In Jetpack Compose, non usiamo variabili normali per i dati che cambiano sullo schermo. Usiamo lo **Stato (`State`)**.

> **Regola d'oro di Compose:** Quando lo *Stato* cambia, Compose "ricompone" (ridisegna) automaticamente tutte le parti della UI che leggevano quello stato.

I poteri magici: `mutableStateOf`

Per trasformare una variabile in uno Stato Osservabile, usiamo la funzione `mutableStateOf()`.

```
// È una variabile speciale: avvisa la UI quando cambia!
val messaggi = mutableStateOf(2)

// Per leggere o modificare il valore reale, usiamo ".value"
messaggi.value = 3
```

I poteri magici: `remember`

C'è un altro problema: se Compose ridisegna lo schermo, rischia di ricreare la variabile da zero (riportandola a 2). Dobbiamo dire all'app di **ricordare** il valore tra un ridisegno e l'altro.

```
// Questa è la formula magica che userete sempre!
var messaggi by remember { mutableStateOf(2) }

// Grazie alla parola "by", non serve più scrivere ".value"
messaggi = 3
```

virvelle

L'Anatomia di un Bottone Interattivo

```
@Composable fun ContatoreMiPiace() { var likes by remember { mutableStateOf(0) }
        Button(onClick = {
    likes++ // Questo aggiorna lo stato!
}) {
  Text("Mi piace: $likes")
        }
}
```

Dibattito: L'Illusione della Memoria

Cosa succede se ruotate fisicamente il vostro smartphone mentre usate l'App "Calcolatrice"?

Il sistema Android distrugge completamente la schermata e la ricrea da zero in formato orizzontale. Se usate solo `remember`*, la vostra calcolatrice tornerà a zero! (Domani vedremo come usare* `rememberSaveable` *per superare questo limite).*

Parte 4: Liste Dinamiche

Addio ScrollView, benvenuto LazyColumn

Il Problema prestazionale di `Column`

Nel Giorno 1 abbiamo usato `Column` per impilare testi.
Ma cosa succede se ho una rubrica con **10.000 contatti** e li metto in una `Column`?

Il telefono proverà a caricare e disegnare tutti i 10.000 elementi contemporaneamente nella RAM.
Risultato: Schermo freezato, app in crash, batteria scarica.

La soluzione: La "Pigrizia" (Lazy)

In informatica, essere pigri è una virtù.
Compose ci offre la `LazyColumn` (Colonna Pigra).

Come funziona?
Se lo schermo può mostrare solo 8 contatti alla volta, la `LazyColumn` disegnerà *solo* quegli 8 contatti. Quando scorrete verso il basso, distrugge i contatti in alto e crea quelli in basso al volo!

La Sintassi della LazyColumn

La `LazyColumn` non accetta direttamente componenti al suo interno, ma richiede blocchi speciali chiamati `item` o `items`.

```
LazyColumn {
    item { Text("Titolo della Lista") }

    item { Text("Primo elemento") }

    item { Text("Secondo elemento") }
}
```

Il vero potere: `items(lista)`

Se abbiamo una lista Kotlin (es. `val nomi = listOf(...)`), possiamo darla in pasto alla `LazyColumn` usando il blocco `items`.

Kotlin

```
val toDos = listOf("Spesa", "Palestra", "Studio")

LazyColumn {
    items(toDos) { task ->
        // Questa riga viene ripetuta per ogni elemento!
        Text(text = "- $task", modifier = Modifier.padding(8.dp))
    }
}
```

Con 5 righe di codice abbiamo creato una lista scorrevole illimitata.

virvelle

Uniamo i puntini: Creare una Riga Custom

Invece di mostrare un semplice testo, possiamo creare una "Card" personalizzata per ogni elemento della lista.

Kotlin

```
@Composable
fun TaskRiga(nomeTask: String) {
    Card(modifier = Modifier.padding(8.dp)) {
        Row(modifier = Modifier.fillMaxWidth()) {
            Icon(Icons.Default.CheckCircle, "Fatto")
            Spacer(Modifier.width(10.dp))
            Text(nomeTask, fontSize = 20.sp)
        }
    }
}
```

Inserire la Card nella LazyColumn

```
LazyColumn { items(toDos) { task -> // Richiamo il MIO componente! TaskRiga(nomeTask = task) } }
```

L'architettura moderna è come i LEGO: costruisci un mattoncino piccolo e lo riusi infinite volte nel mattoncino grande.

virvelle

Mettiamoci alla prova!

Kahoot Time 🕹️

Inquadrate il QR Code. Scopriamo chi ha capito la differenza tra Logica, UI e Stato!

Kahoot Time!

PIN: 696660

virvelle

Parte 5: Verso la Pratica

Il progetto To-Do List

Cosa costruiamo oggi?

Nella demo e nel laboratorio pomeridiano costruiremo una **To-Do List** completa.

Funzionalità:

1. Campo di testo per scrivere un nuovo impegno.
2. Bottone "Aggiungi".
3. Lista scorrevole che mostra gli impegni.
4. Possibilità di cliccare un impegno per cancellarlo.

Sfida: Lo Stato di una Lista

Per fare in modo che l'interfaccia si aggiorni quando aggiungiamo un elemento, una normale `MutableList` non basta (Compose non se ne accorgerebbe).

Dobbiamo usare una lista "Speciale" di Compose:

```
// Questa lista avvisa la UI ogni volta che aggiungi/rimuovi qualcosa!
val listaTask = remember { mutableStateListOf<String>() }
```

Architettura dell'App

I 3 blocchi del nostro codice:

1. **L'Input:** Un `TextField` (casella di testo) e un `Button`.
2. **Il Cervello:** La `mutableStateListOf` che conserva le parole.
3. **L'Output:** La `LazyColumn` che disegna la lista aggiornata.

Anteprima della Casella di Testo (Input)

Prima di aprire Android Studio, guardiamo il componente per far scrivere l'utente:

Kotlin

```
var testoInput by remember { mutableStateOf("") }

OutlinedTextField(
    value = testoInput,
    onValueChange = { nuovoTesto -> testoInput = nuovoTesto },
    label = { Text("Nuova attività...") }
)
```

Notate come la casella di testo abbia il proprio Stato per ricordarsi cosa state digitando sulla tastiera!

La logica del Bottone Aggiungi

Quando l'utente preme il bottone, dobbiamo prendere il `testoInput` e spingerlo dentro la nostra `listaTask`.

Kotlin

```
Button(
    onClick = {
        if (testoInput.isNotBlank()) {
            listaTask.add(testoInput) // 1. Aggiungo alla lista
            testoInput = ""           // 2. Svuoto la casella di testo
        }
    }
) {
    Text("Aggiungi")
}
```

Cancellare un elemento

Per cancellare un task, possiamo aggiungere un Modifier `.clickable` alla nostra singola riga.

```
Text(
    text = taskCorrente,
    modifier = Modifier.clickable {
        listaTask.remove(taskCorrente)
    }
)
```

Magia di Compose: appena chiamate `remove`, la riga scompare dallo schermo istantaneamente con una fluida animazione!

Breve Demo 💻

(Condivido lo schermo: assembliamo i pezzi visti nelle slide e lanciamo la To-Do List sull'emulatore!)

virvelle

virvelle

Learn Your Way, Grow With Us.

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary